

Design and Implementation of a 4 Bit CLA

Siddarth Gottumukkula

2023102040

Department of Electronics and Communication Engineering

IIIT Hyderabad

siddarth.g@students.iiit.ac.in

Abstract—This project focuses on the design and implementation of a 4-bit Carry Look-Ahead Adder (CLA), a high-performance digital circuit for binary addition. The CLA architecture is chosen for its ability to significantly reduce propagation delay compared to traditional ripple carry adders, making it suitable for high-speed arithmetic operations in modern digital systems. Performance metrics, such as delay, power consumption, gate count, are evaluated to demonstrate the efficiency of the CLA. Data collection and simulations have been done using tools like NGSPICE, MAGIC layout tool and AMD Xilinx Vivado. This project aims to provide insights into efficient adder design techniques.

I. INTRODUCTION

Addition is a fundamental operation in digital systems, forming the basis of arithmetic and logic computations in processors, ALUs, and other hardware components. While traditional ripple carry adders (RCAs) offer a straightforward implementation, their performance is hindered by carry propagation delays, which grow linearly with the number of bits. To address this bottleneck, the Carry Look-Ahead Adder (CLA) employs a parallelized approach to generate and propagate carry signals, enabling faster computation (the worst case delay remains the same in both the implementations but the performance is much better on average). By precomputing carry signals based on input bits and leveraging efficient logic design, the CLA minimizes delay, making it a preferred choice for high-speed arithmetic operations.

A. Carry Look-Ahead Logic

The CLA uses the concepts of *generate* and *propagate* signals to precompute carry signals [2]. For two input bits A_i and B_i , and the carry-in C_{i-1} for bit i (The first bit will have a C_{in} and the final carry of the operation will be C_3):

- **Generate Signal (G_i):** The generate signal determines if a carry will be generated at bit i , regardless of the input carry.

$$G_i = A_i \cdot B_i$$

- **Propagate Signal (P_i):** The propagate signal indicates if a carry from the previous stage will propagate through bit i .

$$P_i = A_i \oplus B_i$$

- **Carry-Out (C_i):** The carry-out for each bit is computed using the generate and propagate signals:

$$C_i = G_i + (P_i \cdot C_{i-1})$$

1) *Carry Signal Evaluation:* Using the carry-out equation recursively, the carry signals can be computed for all 4 bits as follows:

$$C_0 = G_0 + (P_0 \cdot C_{in})$$

$$C_1 = G_1 + (P_1 \cdot C_0) = G_1 + P_1 \cdot (G_0 + P_0 \cdot C_{in})$$

$$C_2 = G_2 + (P_2 \cdot C_1) = G_2 + P_2 \cdot (G_1 + P_1 \cdot (G_0 + P_0 \cdot C_{in}))$$

This recursive method eliminates the need for sequential carry propagation, enabling parallel computation of carry signals.

2) *Sum Signal Evaluation:* Once the carry signals are determined, the sum for each bit is calculated using the XOR operation:

$$S_i = P_i \oplus C_{i-1}$$

The optimizations used and implementation details are discussed in this project.

B. Integration of Flip-Flops for Timing Stability

To ensure robust operation and eliminate timing issues, flip-flops are integrated at both the input and output stages of the Carry Look-Ahead Adder (CLA). This design choice ensures proper synchronization of signals and prevents glitches or metastability that could occur due to timing mismatches or high-frequency operations.

By clocking both the input and output flip-flops, the circuit aligns its operations with the system clock, eliminating race conditions and ensuring consistent timing. Output flip-flops store the final results of the CLA computation, ensuring that the outputs are held steady until the next clock cycle. This prevents premature changes in the outputs during high-speed operations.

The flip-flops are strategically placed to encapsulate the CLA logic. Each input signal (A_i , B_i , and C_0) is fed into a D flip-flop, clocked by a common system clock. This ensures that all input signals are synchronized before being processed by the CLA logic. The computed carry (C_3) and sum bits (S_0, S_1, S_2, S_3) are captured by another set of D flip-flops at the output. This locks the results and prevents any glitches from propagating downstream.

II. FLIP-FLOP DESIGN

In this project, a True Single-Phase Clock (TSPC) D-Flip-Flop is employed for input and output synchronization in the Carry Look-Ahead Adder (CLA) circuit. The TSPC flip-flop is chosen for its simplicity, low power consumption, less area, and ability to operate at high frequencies, making it an ideal choice for high-speed digital systems [3].

A. Structure of the TSPC D-Flip-Flop

The TSPC D-Flip-Flop operates with a single clock phase, avoiding the need for complementary clock signals, which simplifies clock distribution. It consists of two main stages, a low level sensitive latch followed by a high level sensitive latch.

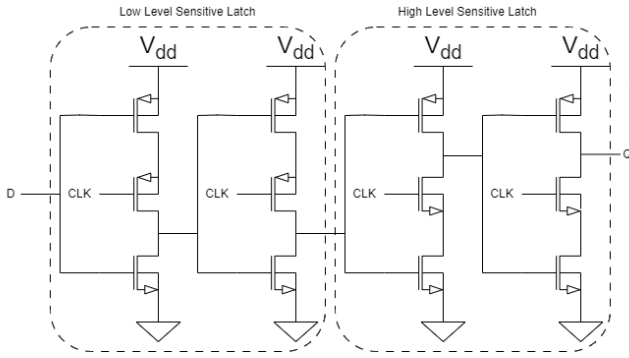


Fig. 1. TSPC D Flip-Flop circuit

The TSPC flip-flop's operation is divided into two phases:

- 1) During the clock's low phase ($CLK = 0$), the low level sensitive latch records the input but the output does not change as the high level sensitive latch is turned OFF.
- 2) During the clock's high phase ($CLK = 1$), the low level sensitive latch turns OFF and the high level sensitive latch turns ON sending the previously recorded input to the output. Any change in the input in this phase will not affect the output.

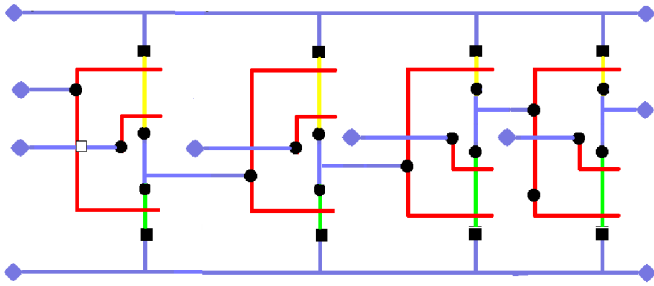


Fig. 2. Stick Diagram of the TSPC D FLip Flop

B. Timing Constraints

The TSPC D-Flip-Flop introduces certain timing constraints that must be satisfied for reliable operation:

- **Setup Time (T_{setup}):** The minimum time the input data (D) must be stable before the clock edge.

- **Hold Time (T_{hold}):** The minimum time the input data (D) must remain stable after the clock edge.
- **Clock-to-Q Delay (T_{CQ}):** The time it takes for the output (Q) to reflect a change in the input (D) after the clock edge.

The following table summarizes the relevant parameters of the TSPC D-Flip-Flop used in this project:

The optimizations used are listed below.

- 1) To reduce the body effect which might adversely affect the circuit, we short the bulk terminal to the source, instead of connecting it to the highest or the lowest voltage terminal in the circuit. This sets $V_{SB} = 0$ and the threshold voltage remains unchanged in our analysis.
- 2) The widths of the PMOS transistors are scaled to twice the widths of the NMOS transistors in order to match the current carrying capabilities. This minimizes the difference in the rise and fall delays.

TABLE I
SIZING USED IN THE TSPC D FF, $\lambda = 0.09\mu m$

Element	PMOS	NMOS
Low Level Latch	40λ	10λ
High Level Latch	20λ	20λ

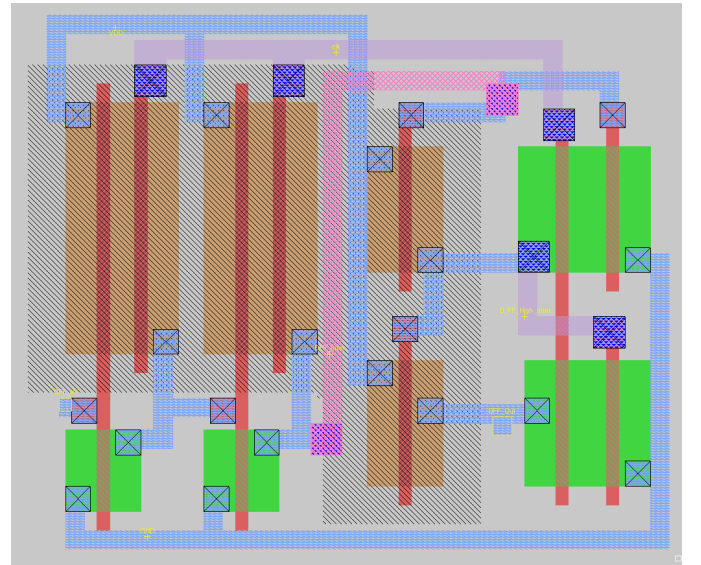


Fig. 3. Layout of TSPC D FF

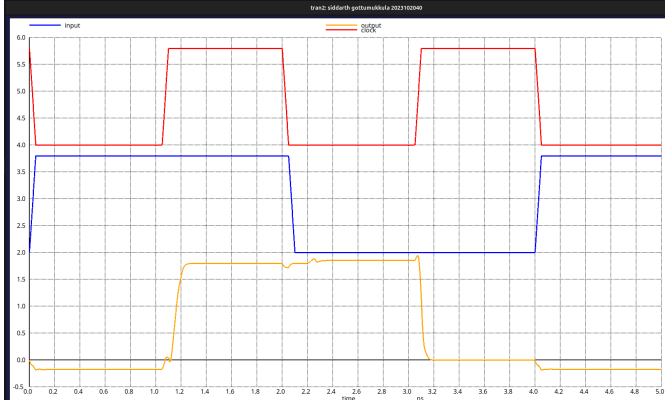


Fig. 4. Pre Layout Output of the FF

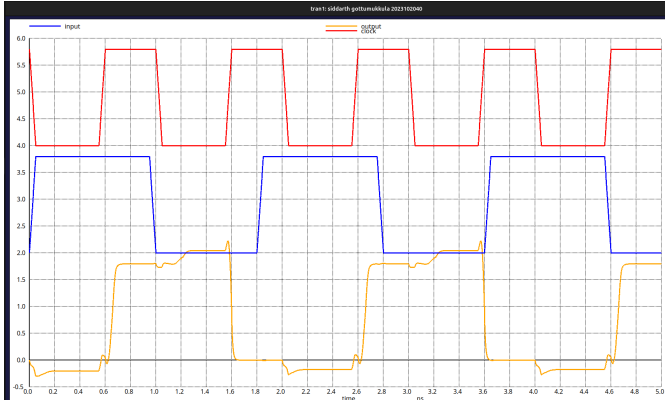


Fig. 5. Post Layout Output of the FF

The Flip-Flop was simulated using NGSPICE pre layout and post layout. The layout was done using MAGIC Layout tool. We expect differences in the values obtained pre layout and post layout not only due to parasitic capacitances present but also because of layout optimization techniques used. For example, the same diffusion region can act as the drain and source of two transistors instead of a wire connecting two separate regions. This cumulatively saves a huge amount of silicon area.

TABLE II
TSPC D-FLIP-FLOP PERFORMANCE DATA

Parameter	Pre Layout	Post Layout
Setup Time (T_{setup})	121.97 ps	129.57 ps
Hold Time (T_{hold})	0 ps	0 ps
Clock-to-Q Delay ($T_{\text{clk_to_Q}}$)	84 ps	82.7 ps

trise	=	7.835443e-11	targ=	1.207014e-09	trig=	1.128660e-09
tfall	=	4.104000e-11	targ=	3.130091e-09	trig=	3.089051e-09
delay	=	5.96972e-11				
tcqrise	=	8.400291e-11	targ=	1.159003e-09	trig=	1.075000e-09
tcqfall	=	2.868698e-11	targ=	3.103687e-09	trig=	3.075000e-09
tcq	=	5.63449e-11				
tsetuprise	=	1.162454e-10	targ=	6.412454e-10	trig=	5.250000e-10
tsetupfall	=	1.219752e-10	targ=	2.696975e-09	trig=	2.575000e-09
tsetup	=	1.19110e-10				

Fig. 6. Various parameters Pre Layout

trise	=	4.565488e-11	targ=	6.797863e-10	trig=	6.341314e-10
tfall	=	2.271572e-11	targ=	1.612584e-09	trig=	1.589868e-09
delay	=	3.41853e-11				
tcqrise	=	8.271269e-11	targ=	6.577127e-10	trig=	5.750000e-10
tcqfall	=	2.421778e-11	targ=	1.599218e-09	trig=	1.575000e-09
tcq	=	5.34652e-11				
tsetuprise	=	1.115862e-10	targ=	3.865862e-10	trig=	2.750000e-10
tsetupfall	=	1.295769e-10	targ=	1.354577e-09	trig=	1.225000e-09
tsetup	=	1.20582e-10				

Fig. 7. Various parameters Post Layout

III. CLA IMPLEMENTATION

The implementation of the Carry Look-Ahead Adder (CLA) in this project focuses on achieving high-speed, power-efficient, and robust operation. A static CMOS logic style is chosen for the CLA circuit, as it offers several advantages over other logic styles, including dynamic logic and alternative static implementations.

A. Choosing a Logic Style

Static logic styles are preferred over dynamic logic styles in this design for the following reasons:

- **Output Requirements:** Given the constraint that the output must appear before at the next clock edge, there isn't much flexibility to choose dynamic logic styles. Implementing the different sections of the CLA using dynamic logic would take many clock cycles to produce the correct output. One strong contender in dynamic logic was to break the CLA implementation into two stages and use level sensitive logic implementations but it has other intricate complexities coming with it which are explained below.
- **Charge redistribution:** Static logic eliminates the risk of charge leakage and redistribution over time, which can lead to incorrect outputs in dynamic circuits. Solving this issue in dynamic circuits requires additional Keeper circuitry.
- **Power Consumption:** Unlike dynamic logic, static logic does not require precharge and evaluation phases, reducing the overall power consumption in the circuit.

Static CMOS logic is used specifically for the CLA implementation due to its balance of performance, power, and reliability:

- **Low Power Consumption:** CMOS logic consumes power only during transitions, making it highly energy-efficient compared to other static styles like pseudo-NMOS or pass-transistor logic.
- **High Noise Margin:** CMOS gates provide high noise margins, ensuring robust operation even under variations in supply voltage or process parameters.
- **Design Simplicity:** CMOS logic gates are straightforward to design and verify, reducing the complexity of the CLA implementation. item **BiCMOS** technology is another good option. However, circuits employing this style use the different families of transistors to their advantage. The bipolar junction transistors are used to handle the analog section (RF) of the circuit and the

CMOS technology is used to handle the ‘sea of logic’ [4]. Thus, for our purposes, BJT’s aren’t really necessary.

B. Circuit Implementation

The most basic circuit one can make is directly from the CLA expressions discussed in the introductory section, using OR and AND gates and XOR gates. However, we know that implementing a non-inverting logic will take up more resources. A possible implementation is to use NAND and NOR gates, the simplest implementation for which is detailed in [5]. A potential problem with this circuit is that a large number of inputs to a gate will result in a large capacitance, which will make the delay larger. We want to implement a circuit with a smaller parasitic capacitance, i.e., a smaller number of inputs. With a slight manipulation of the logic expressions, we observe that an implementation with just two input gates is possible [1]. We will explore this implementation further.

Another interesting implementation could be generating the output bits using a large single PUN and PDN [6]. However, to sizing of each of the transistors would be significantly higher than the basic implementation. The complexity also increases. Though it may prove to be faster due to lesser number of logic stages the signal has to propagate through, the capacitance and the area would be drastically higher.

Another optimization that can be made to reduce the parasitic capacitance is to primarily use NOR gates instead of NAND gates. This will reduce the capacitance seen by the output node. The reason for this is that the PMOS stack will have a larger capacitance than the NMOS stack due to the sizing. Notice that the XOR in the Propagate signal can be changed to OR without affecting the output. This will reduce the complexity of the circuit. Let us redefine our logic expressions to inculcate these changes.

$$P_i = A_i + B_i \text{ for } i \in \{0, 1, 2, 3\}$$

$$G_i = A_i \cdot B_i \text{ for } i \in \{0, 1, 2, 3\}$$

$$\overline{P_i} = \overline{A_i + B_i} \text{ for } i \in \{0, 1, 2, 3\}$$

$$\overline{G_i} = \overline{A_i \cdot B_i} \text{ for } i \in \{0, 1, 2, 3\}$$

$\overline{P_i}$ is the NOR of the input bits and $\overline{G_i}$ is the NAND of the input bits.

$$\begin{aligned} C_0 &= G_0 + P_0 \cdot C_{in} \\ &= \overline{\overline{G_0} \cdot (\overline{P_0 \cdot C_{in}})} \\ &= \overline{\overline{G_0} \cdot (\overline{P_0} + \overline{C_{in}})} \\ C_1 &= G_1 + P_1 \cdot G_0 + P_1 \cdot P_0 \cdot C_{in} \\ &= \overline{\overline{G_1} \cdot (\overline{P_1 + G_0}) + (\overline{P_1} + \overline{P_0}) \cdot C_{in}} \\ C_2 &= G_2 + P_2 \cdot G_1 + P_2 \cdot P_1 \cdot G_0 + P_2 \cdot P_1 \cdot P_0 \cdot C_{in} \\ &= \overline{\overline{G_2} \cdot (\overline{P_2 + G_1}) + P_2 \cdot P_1 \cdot (G_0 + P_0 \cdot C_{in})} \\ &= \overline{\overline{G_2} \cdot (\overline{P_2} + \overline{G_1}) + (\overline{P_2} + \overline{P_1}) \cdot \overline{G_0 \cdot P_0 \cdot C_{in}}} \\ &= \overline{\overline{G_2} \cdot (\overline{P_2} + \overline{G_1}) + (\overline{P_2} + \overline{P_1}) \cdot \overline{G_0 \cdot P_0} + \overline{C_{in}}} \\ C_3 &= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 + P_3 \cdot P_2 \cdot P_1 \cdot G_0 \\ &\quad + P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot C_{in} \end{aligned}$$

We decompose C_3 into two different intermediate expressions.

$$\begin{aligned} C_3 &= G_{out} + P_{out} \cdot C_{in} \\ &= \overline{\overline{G_{out}} \cdot (\overline{P_{out} + C_{in}})} \\ G_{out} &= G_3 + P_3 \cdot G_2 + P_3 \cdot P_2 \cdot G_1 \\ &\quad + P_3 \cdot P_2 \cdot P_1 \cdot G_0 \\ &= \overline{\overline{G'_3} \cdot (\overline{P'_3 + G'_2})} \\ &\quad + (\overline{P'_3} + \overline{P'_2}) \cdot \overline{G'_1 \cdot (P'_1 + G'_0)} \\ P_{out} &= P_3 \cdot P_2 \cdot P_1 \cdot P_0 \\ &= \overline{\overline{P'_3} + \overline{P'_2} \cdot \overline{P'_1} + \overline{P'_0}} \end{aligned}$$

The stick diagrams of the logic gates used are drawn [8] and the sizing is explained in the later part of this section. The circuit implementing the logic can be designed as shown in [1].

TABLE III
LEGEND FOR THE STICK DIAGRAMS

Element	Color/Symbol
Metal 1	Blue
P Diffusion	Yellow
N Diffusion	Green
Polysilicon	Red
Contact	Black Circle
Contact & Tap	Black Square
Metal 1 Port	Blue Diamond

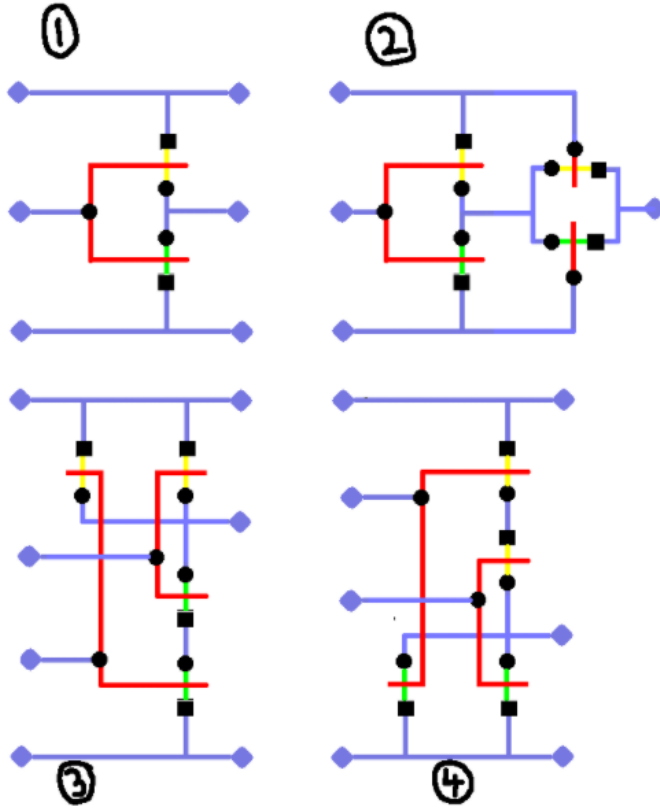


Fig. 8. Stick Diagrams of the gates NOT, XOR [9], NAND, NOR. These structures show how the transistors are connected in NGSPICE and MAGIC (for NOT and XOR). The color mapping is given below.

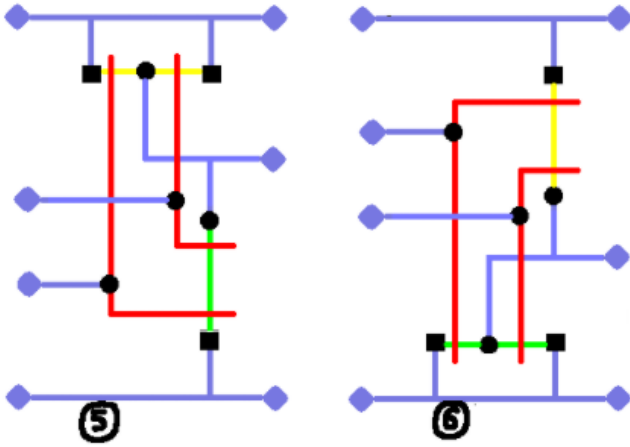


Fig. 9. In MAGIC, the common terminals are connected as shown for NAND and NOR gates. This gives us different results than the expected values from NGSPICE

To decide the sizing of the gates, we can use the method of logical effort [7]. The circuit, however, is too complex with multiple paths. Optimizing for a critical path might change the critical path which we will have to iterate over again. Thus, I have used a few general inferences to improve the circuit. For instance, the NAND and NOR gates which generate $\bar{G}_i$ and $\bar{P}_i$ are made larger to increase the driving capability. This

trend can be seen from MLE as well. The logic gates in the middle section which compute the carry are sized smaller than the first layer of gates and the logic gates towards the end, before the output, are the smallest. This ensures less load and optimal current driving capabilities for each of the stages. The gate sizes are given in the table ?? and circuit diagram ?? has numbers on the logic gates indicating the types of gates used.

TABLE IV
SIZING USED IN THE TSPC D FF, $\lambda = 0.09\mu m$

Element	PMOS	NMOS	Numbering
Inverter	22λ	11λ	1
Small NOR	10λ	5λ	2
Medium NOR	40λ	20λ	3
Large NOR	80λ	40λ	4
Small NAND	20λ	20λ	5
Large NAND	40λ	40λ	6

The XOR gate's (7) Inverter part is made using the same parameters specified above. The analog transmission gate is made using $W_p = 21\lambda$, $W_n = 5\lambda$, these are the values that just satisfy all the DRCs. The AND gate (8) is made using the NAND gate and the Inverter. The OR gate (9) is made using the Medium NOR gate and the inverter. The values were initialized in an attempt to match the resistance to an inverter and later iterated over to find the current set of values.

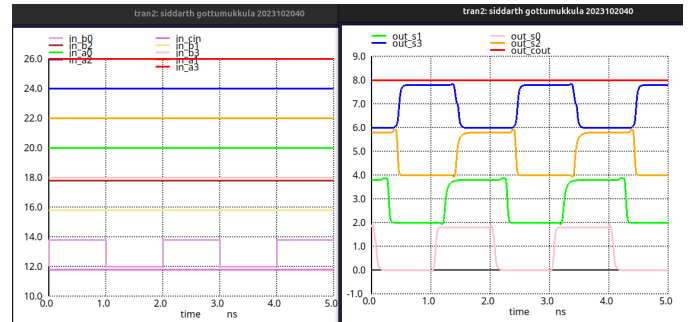


Fig. 10. CLA Pre Layout Output for the case, $A = 0000$, $B = 0110 \rightarrow 0111$, $C = 1$. This is a case which tests the critical path from B_0 to S_3

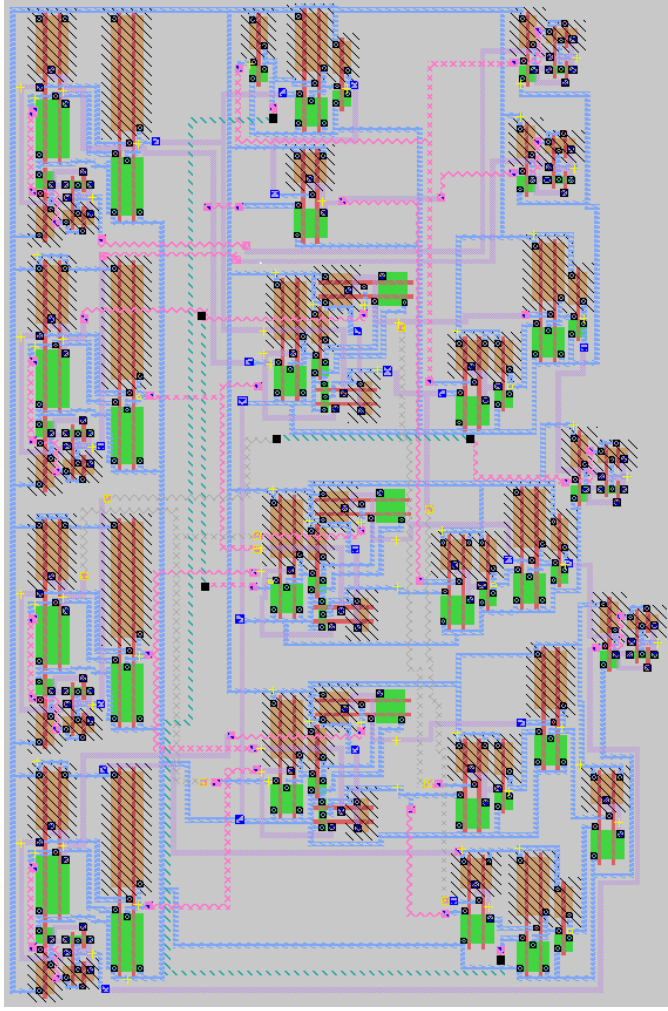


Fig. 11. CLA Layout

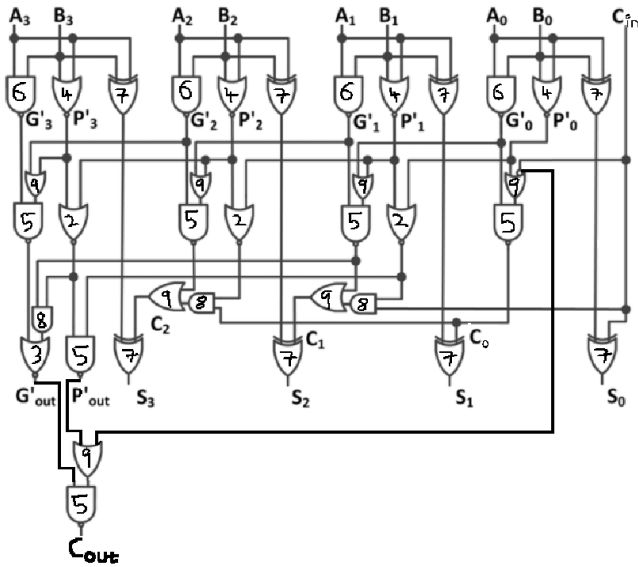


Fig. 12. Circuit Diagram [1] with gates numbered as mentioned in the table above

The critical path in the circuit would be one from B_0 to S_3 . The critical path can be figured out by counting the number of gates in the path. We will take a test case and analyze the same test case in all the simulations unless stated otherwise.

The circuit can be broken down and implemented in three layers. The first layer generates the $\bar{P}$ and $\bar{G}$ signals along with a partial Sum bit signal. The next layer will involve intermediate computation of the carry bits and the final layer will compute the sum bit and produce the output ultimately.

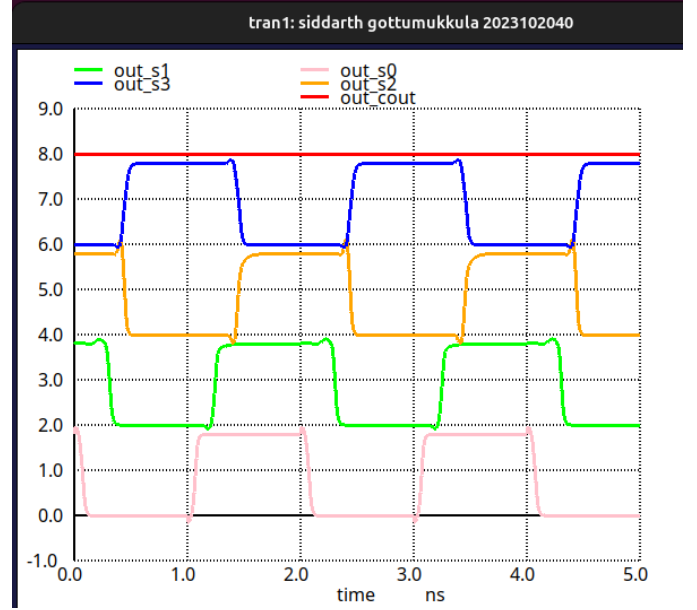


Fig. 13. CLA Post Layout Output for the same case

TABLE V
PARAMETERS OF THE 4-BIT CARRY LOOKAHEAD ADDER

Parameter	Value
Pre-Layout Propagation Delay (ns)	459.98 ps
Post-Layout Propagation Delay (ns)	449.57 ps
Pre-Layout Power Delay Product (PDP)	20.7 pJ
Post-Layout Power Delay Product (PDP)	20.23 pJ
Power Dissipation (mW)	45 mW
Gate Count (Layout)	54
Transistor Count (Layout)	182
Layout Area	2585.73 μm^2
Horizontal Pitch	41.22 μm
Vertical Pitch	62.73 μm

trise	=	6.922182e-11	targ=	4.994074e-10	trig=	4.301856e-10
tfall	=	9.650383e-11	targ=	1.491623e-09	trig=	1.395119e-09
delay	=	8.28628e-11				
tpdrise	=	4.599855e-10	targ=	4.604855e-10	trig=	5.000000e-13
tpdfall	=	4.518243e-10	targ=	1.453324e-09	trig=	1.001500e-09
tpd	=	4.55905e-10				

Fig. 14. Parameters Pre Layout for CLA

```

trise      = 6.901737e-11 targ= 4.827477e-10 trig= 4.137303e-10
tfall      = 6.346645e-11 targ= 1.479898e-09 trig= 1.416432e-09
delay      = 6.62419e-11
tpdrise    = 4.420269e-10 targ= 4.425269e-10 trig= 5.000000e-13
tpdfall    = 4.495703e-10 targ= 1.451070e-09 trig= 1.001500e-09
tpd        = 4.45799e-10

```

Fig. 15. Parameters Post Layout for CLA

Theoretically, we can come up with a bound for the maximum frequency using **Static Timing Analysis**.

$$t_{CQ_{max}} + t_{pd_{max}} + t_{su} \leq T_{Clk}$$

Using the above equation and the values extracted earlier, we get,

$$f_{maxPrelayout} = 1.502 \text{ GHz}$$

$$f_{maxPostlayout} = 1.511 \text{ GHz}$$

We will compare the actual frequency of operation obtained with this value later on.

IV. INTEGRATION

Floor planning has to be proper to ensure optimal circuit area.

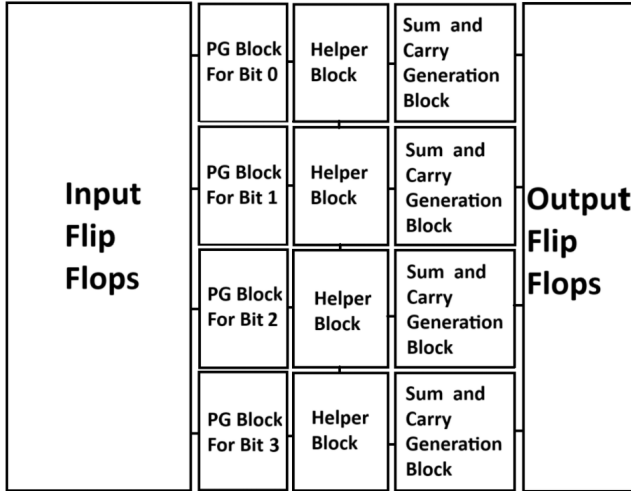


Fig. 16. Floor Plan for the Layout. The circuit is divided into these sub blocks and implemented for modularity

The circuit designing and performance metrics are specified below. They are extracted with an inverter $\left(\frac{20\lambda}{10\lambda}\right)$ as load at each of the flip flop outputs.

TABLE VI
PRE-LAYOUT VS POST-LAYOUT PARAMETERS OF THE 4-BIT CLA

Parameter	Value
Circuit Area	4477.81 μm^2
Transistor Count	360
Horizontal Pitch	62.82 μm
Vertical Pitch	71.28 μm
Max Frequency Pre-Layout	1.136 GHz
Min Time Period Pre-Layout	0.88 ns
Max Frequency Post-Layout	1.19 GHz
Min Time Period Post-Layout	0.84 ns

We see that the maximum frequency of operation is **0.3 GHz lesser** than what is expected theoretically. This arises due to various parameters such as loading and parasitics.

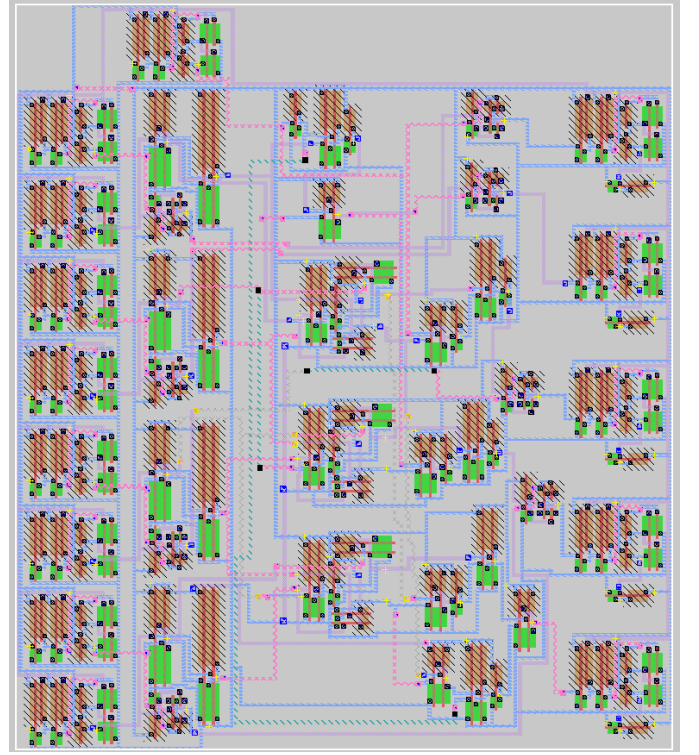


Fig. 17. Full Circuit Layout

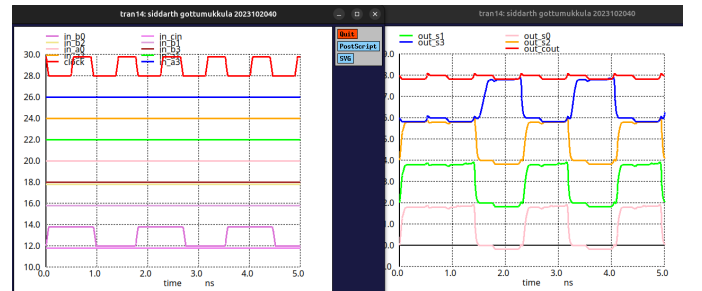


Fig. 18. Pre Layout Circuit operating at 1.136 GHz for the same critical case

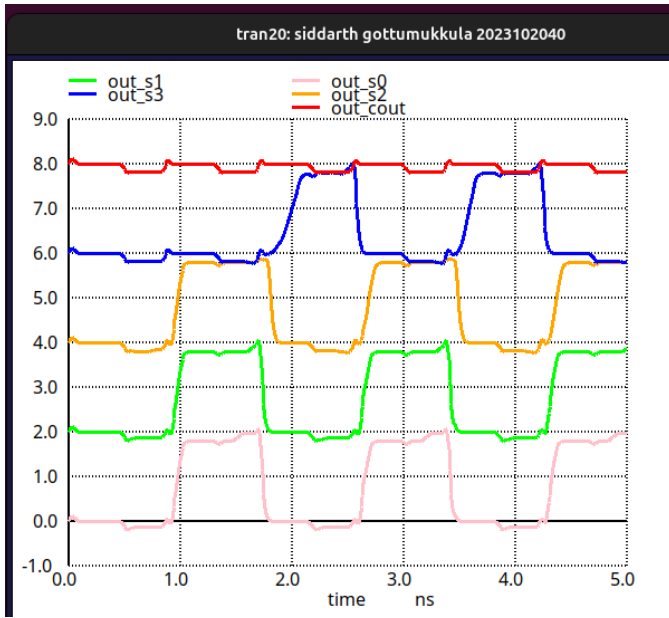


Fig. 19. Post Layout circuit operating at 1.19 GHz

We can verify the timing of the input and the output using Verilog. In structural level code, we need not worry about the sizing or voltage levels. It is purely for logic simulation and timing analysis which gives us insights into the working of the circuit.

Implementation has been carried out on the Xilinx Spartan-7 Boolean FPGA board. Using the PMOD pins, the output has been plotted on an Oscilloscope.

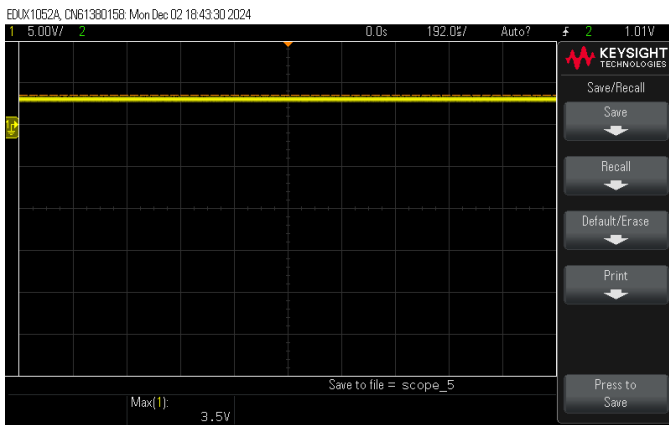


Fig. 21. Carry Bit for the input A = 13 (1011), B = 7 (0111), $C_{in} = 1$. The FPGA board outputs values between 0 V and 3.3 V through the PMOD pins. In the oscilloscope plots, 3.3 V is to be treated as logic level 1 and 0 is supposed to be treated as logic level 0. However, values observed are not exact due to noise. Repeaters can be added to tackle this issue. An other solution is to make the circuit following this circuit robust to noise by ensuring a good noise margin.

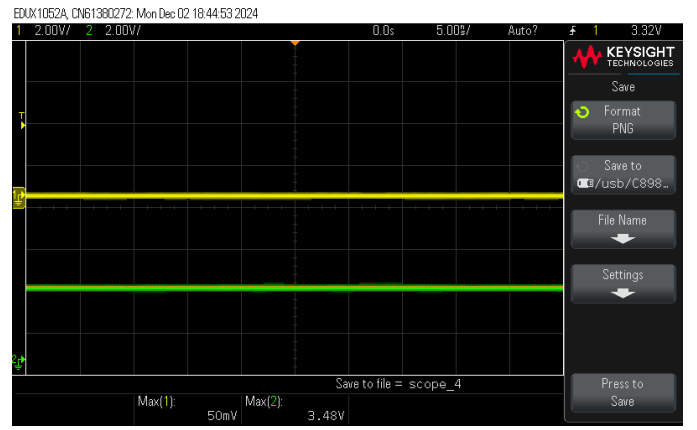


Fig. 22. S_3 and S_2 bits for the input A = 13 (1011), B = 7 (0111), $C_{in} = 1$

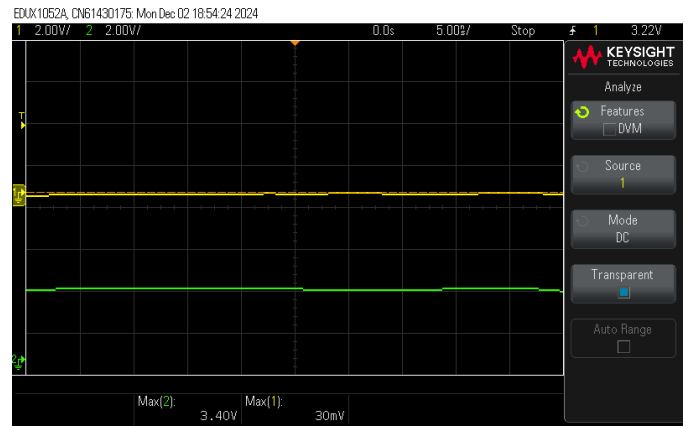


Fig. 23. S_0 and S_1 bits for the input A = 13 (1011), B = 7 (0111), $C_{in} = 1$

The final output of the case A = 13 (1011) + B = 7 (0111) + $C_{in} = 1 = 21$ (10101)

Thus, with various parameters extracted above in line with theory and simulations run on both software and hardware, we can claim that the design was successful. There is quite some scope for improvement. The area can be optimized, the sizing can be tweaked, or the entire logic style can be changed for better performance!

ACKNOWLEDGMENT

I would like to express my sincere gratitude to Professor Abhishek Srivastava, whose guidance and expertise have been invaluable throughout this project. Their insightful lectures and encouragement inspired me to delve deeper into the design and implementation of the Carry Lookahead Adder.

I am also deeply thankful to the Teaching Assistants, Karthikeya and Rathan, for their support and for addressing my queries promptly. Their feedback greatly contributed to the successful completion of this project.

REFERENCES

- [1] J. Miao and S. Li, "A novel implementation of 4-bit carry look-ahead adder", 2017 International Conference on Electron Devices and Solid-State Circuits (EDSSC), Hsinchu, Taiwan, 2017, pp. 1-2.

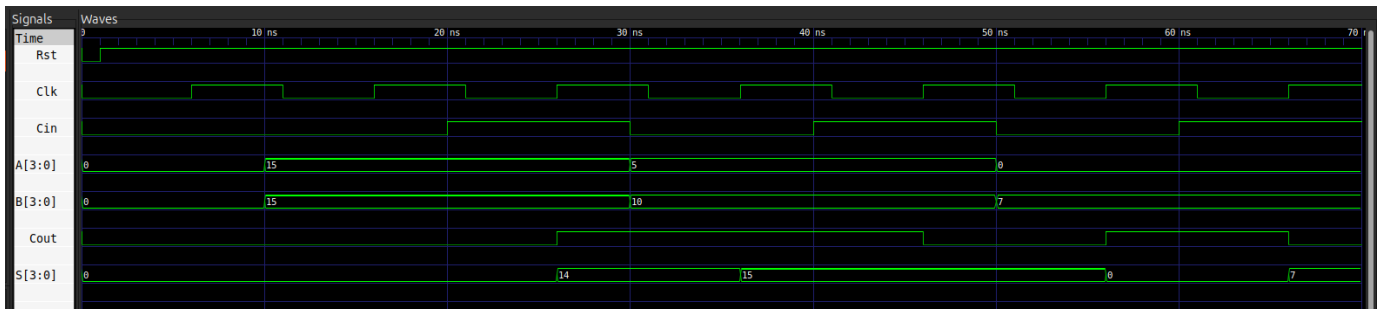


Fig. 20. Verilog output

- [2] Mano, M. Morris, and Michael D. Ciletti. Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog. 5th edition. Pearson, 2012.
- [3] B. Razavi, "TSPC Logic [A Circuit for All Seasons]", in IEEE Solid-State Circuits Magazine, vol. 8, no. 4, pp. 10-13, Fall 2016.
- [4] "BiCMOS", Wikipedia, <https://en.wikipedia.org/wiki/BiCMOS>.
- [5] Yu-Ting Pai and Yu-Kung Chen, "The fastest carry lookahead adder," Proceedings. DELTA 2004. Second IEEE International Workshop on Electronic Design, Test and Applications, Perth, WA, Australia, 2004, pp. 434-436.
- [6] M. Hasan, P. Biswas, M. S. Alam, H. U. Zaman, M. Hossain and S. Islam, "High Speed and Ultra Low Power Design of Carry-Out Bit of 4-Bit Carry Look-Ahead Adder," 2019 10th International Conference on Computing, Communication and Networking Technologies (ICCCNT), Kanpur, India, 2019, pp. 1-5.
- [7] Ivan E. Sutherland, Bob F. Sproull and David L. Harris. Logical Effort: Designing Fast CMOS Circuits, Morgan Kaufmann Publishers, 1998.
- [8] Drawing Stick Diagrams, Southampton University, <https://www.southampton.ac.uk/bim/notes/cad/guides/sticks.html>.
- [9] NS-SRAM: Neighborhood Solidarity SRAM for Reliability Enhancement of SRAM Memories - Scientific Figure on ResearchGate. Available from: https://www.researchgate.net/figure/mplementation-of-XOR-gate-with-four-transistors_fig3_304142640.